Silicon Misiones
Nuestra naturaleza es evolucionar

PROYECTO DEL MÓDULO 3

PRIMER TRABAJO ENTREGABLE · CORRECCIÓN AUTOMÁTICA

API de la Biblioteca

Una API REST con FastAPI para gestionar libros

BOOTCAMP
Silicon Misiones ·

DISPONIBLE DESDE
Lunes 9 de noviembre de 2026

FECHA LÍMITE
Lunes 16 de noviembre de 2026 · hasta las **19:30 hs**

MEDIO DE ENTREGA
Repo público en GitHub + formulario en **edemake.com/entrega-backend-integrador-1**

MODALIDAD
Individual, fuera de clase

CON QUÉ SE HACE
Solo lo visto en las Clases 1 a 22

SE CORRIGE CON
Un **programa automático**

1 · Qué vas a construir y cómo se corrige

Una **API REST con FastAPI** para gestionar los libros de una biblioteca: crear, listar, actualizar y borrar; y **seleccionar partes** de esos datos (filtrar, paginar, buscar) y devolverlas. Los datos se **guardan en un archivo JSON** (Clase 15) y sobreviven al reinicio: la persistencia es **obligatoria**. Es la estructura de la API de tareas de las Clases 18 a 22, aplicada a **otro recurso**.

La corrección no la hace una persona leyendo tu código: la hace **un programa**. Ese programa levanta tu API, le hace pedidos (igual que vos en `/docs`) y compara cada respuesta con lo esperado: la ruta, el código de estado, los nombres de los campos y los valores. Por eso este documento define un **contrato**: cliente y servidor se ponen de acuerdo en los nombres exactos. Si tu API cumple el contrato, la prueba pasa. Si no, falla, aunque "funcione".

⚠️ **Un nombre de campo mal escrito significa que esa prueba falla.**
`anio` es `anio`, no `año`. `titulo` es `titulo`, no `título`. `/libros` es `/libros`, no `/libro`. Copiá los nombres de este documento, no los escribas de memoria.

— 2 · Cómo se entrega

- [ ] **1.** Subí el proyecto a un **repositorio de GitHub** (Clase 16). El repositorio debe ser **público**: si es privado, no se puede revisar.
- [ ] **2.** Completá el formulario en **`edemake.com/entrega-backend-integrador-1`** con: **DNI**, **correo electrónico**, **nombre y apellido completos**, **URL del repositorio** y observaciones (opcional).
- [ ] **3.** Con el envío del formulario, al profe le llega por correo la solicitud de revisión. No hace falta avisar por otro lado.
- [ ] **4.** Se corrige la **rama principal** del repositorio, con el **último commit hasta la fecha y hora límite**. Lo que se suba después no se tiene en cuenta.



La URL es la del repositorio, no la del perfil. `github.com/tu-usuario/api-biblioteca` ✓ · `github.com/tu-usuario` ✗. Antes de enviarla, **abrila en una ventana de incógnito**: si la ves sin iniciar sesión, el profe también.

SI TE EQUIVOCASTE AL COMPLETAR EL FORMULARIO · COMPLETA EL PROFE

_____

CONDICIONES TÉCNICAS DEL REPOSITORIO

- [ ] `main.py` está en la **raíz** del repositorio, con una variable `app` (`app = FastAPI()`) que se pueda **importar**. Podés usar más archivos (schemas.py, datos.py…), pero `main.py` debe exponer `app`.
- [ ] Importar `main.py` **no ejecuta nada más**: sin `input()`, sin bucles infinitos, sin `uvicorn.run` suelto. El servidor lo levanta el corrector.
- [ ] Incluye `requirements.txt`, `README.md` y `.gitignore`.
- [ ] **No** se suben `.venv`, `venv` ni `__pycache__`.
- [ ] Solo **fastapi, pydantic, uvicorn** y la **biblioteca estándar**.

ESTRUCTURA MÍNIMA DEL REPOSITORIO

```
📁 api-biblioteca/
├─ main.py  ← con app = FastAPI()
├─ requirements.txt
├─ README.md
├─ .gitignore
├─ 📁 datos/  ← libros.json se crea acá
└─ (opcional) schemas.py · datos.py
```

3 · El contrato de la API

El recurso es **libro**. Tiene **exactamente** estos cinco campos; las respuestas no deben traer ni más ni menos.

CAMPO	TIPO	REGLAS	¿SE ENVÍA?
`id`	entero	Lo asigna el servidor. Empieza en 1, sube de a uno, **no se reutiliza**. Tras reiniciar: mayor id guardado + 1.	**No. Solo en respuestas.**
`titulo`	texto	De **3 a 100** caracteres. Sin tilde en el nombre del campo.	**Sí, obligatorio.**
`autor`	texto	De **3 a 60** caracteres.	**Sí, obligatorio.**
`anio`	entero	Entre **1000 y 2100**. Escrito **sin ñ**.	**Sí, obligatorio.**
`disponible`	booleano	Por defecto `true`.	**Opcional.**

Entrada (POST y PUT): `titulo`, `autor`, `anio` y, opcional, `disponible`. **Salida:** los mismos cuatro **más** `id`.

Las **listas** van ordenadas por `id` de menor a mayor. Sin archivo de datos, la API arranca **vacía**. Los errores **404 y 409** devuelven `{"detail": "…"}` (texto libre). El **422** lo genera FastAPI solo.

ENDPOINTS DEL CONTRATO

MÉTODO	RUTA	RECIBE	DEVUELVE	CÓDIGOS
GET	`/`	—	`{"mensaje": "API de la biblioteca"}`	**200**
GET	`/libros`	filtros y paginación opcionales (ver I3, I4)	lista de libros (puede estar vacía)	**200**
GET	`/libros/buscar`	?texto=... (obligatorio)	lista de libros que coinciden	**200** · 422
POST	`/libros`	body con titulo, autor, anio, disponible?	el libro creado, con id	**201** · 409 · 422
GET	`/libros/{id}`	id entero en la ruta	el libro	**200** · 404 · 422
PUT	`/libros/{id}`	body completo (reemplaza todo)	el libro reemplazado	**200** · 404 · 422
PATCH	`/libros/{id}`	body parcial (solo lo que cambia)	el libro completo actualizado	**200** · 404 · 422
DELETE	`/libros/{id}`	id entero en la ruta	**nada** (sin cuerpo)	**204** · 404

+ los endpoints opcionales de la sección 7: `/libros/{id}/prestar`, `/libros/{id}/devolver`, `/libros/estadisticas`.

EJEMPLO · POST /libros

```
# pedido (body)
{ "titulo": "Rayuela",
  "autor": "Julio Cortázar",
  "anio": 1963 }

# respuesta · 201 Created
{ "id": 1,
  "titulo": "Rayuela",
  "autor": "Julio Cortázar",
  "anio": 1963,
  "disponible": true }
```

EJEMPLO · GET /libros (con dos libros)

```
# respuesta · 200 OK
[
  { "id": 1, "titulo": "Rayuela",
    "autor": "Julio Cortázar",
    "anio": 1963, "disponible": true
},
  { "id": 2, "titulo": "El Aleph",
    "autor": "Jorge Luis Borges",
    "anio": 1949, "disponible": false
}
]
# ordenada por id, menor a mayor
```

Pista de orden: las rutas fijas como `/libros/buscar` y `/libros/estadisticas` deben declararse **antes** que `/libros/{id}`. Si no, FastAPI intenta convertir "buscar" a entero y devuelve 422.

4 · Tareas FÁCILES **40 puntos**

Son la base. **Completar todas** es la condición para aprobar. Cada una se comprueba con una prueba automática.

- [ ] **F1** **4 pts**
`GET /` devuelve **200** y exactamente `{"mensaje": "API de la biblioteca"}`.

- [ ] **F2** **4 pts**
`GET /libros` devuelve **200** y una **lista**. Sin archivo de datos, la lista está vacía: `[]`.

- [ ] **F3** **8 pts**
`POST /libros` con datos válidos devuelve **201** y el libro creado con `id: 1`. Si no se envía `disponible`, en la respuesta vale `true`. *Ejemplo: el de la sección 3.*

- [ ] **F4** **8 pts**
`GET /libros/{id}` devuelve **200** y el libro correcto (mismos campos y valores que al crearlo). *Ejemplo: tras crear Rayuela, `GET /libros/1` devuelve Rayuela.*

- [ ] **F5** **8 pts**
(4) `GET /libros/999` (id inexistente) devuelve **404** con `{"detail": …}`. **(4)** `GET /libros/abc` (id que no es número) devuelve **422**.

- [ ] **F6** **8 pts**
(4) `DELETE /libros/{id}` de un libro existente devuelve **204 sin cuerpo**, y después `GET /libros/{id}` de ese mismo id devuelve **404**. **(4)** `DELETE` de un id inexistente devuelve **404**.

5 · Tareas INTERMEDIAS **50 puntos**

- [ ] **I1** **8 pts** · Validación con Pydantic
`POST /libros` devuelve **422** cuando: el `titulo` tiene menos de 3 caracteres · tiene más de 100 · falta el `autor` · el `anio` es menor que 1000 · mayor que 2100 · no es un número (ej. `"abc"`). *Aclaración:* un número escrito como texto (`"2000"`) puede ser aceptado; eso no se evalúa.

- [ ] **I2** **6 pts** · PUT

`PUT /libros/{id}` **reemplaza todos** los campos con los enviados (si no se envía `disponible`, queda `true`) y devuelve **200** con el libro actualizado, con el mismo `id`. **404** si no existe. **422** si los datos son inválidos (mismas reglas que I1).

- [ ] **I3** **5 pts** · Filtros

`GET /libros?autor=texto` devuelve solo los libros cuyo autor coincide **exactamente**, **sin distinguir mayúsculas y minúsculas** (`?autor=julio cortázar` encuentra a "Julio Cortázar"). `?disponible=true` o `false` filtra por disponibilidad. **Se pueden combinar**: `?autor=…&disponible=true`.

- [ ] **I4** **5 pts** · Paginación

`GET /libros?desde=N` saltea los primeros N libros (por defecto **0**); `?limite=N` devuelve como máximo N (por defecto **100**). **Primero se aplican los filtros, después la paginación.** *Ejemplo: con 5 libros, `?desde=2&limite=2` devuelve los de id 3 y 4.*

- [ ] **I5** **5 pts** · PATCH

`PATCH /libros/{id}` actualiza **solo los campos enviados** y deja el resto igual; devuelve **200** con el libro completo. Un cuerpo vacío `{}` devuelve **200** sin cambios. **404** si no existe. **422** si algún campo enviado es inválido (mismas reglas). *Ejemplo: `{"disponible": false}` cambia solo eso.*

- [ ] **I6** **3 pts** · Conflicto 409

`POST /libros` devuelve **409** con `{"detail": …}` si ya existe un libro con el **mismo titulo y el mismo autor**, sin distinguir mayúsculas y minúsculas. (`"rayuela"` + `"JULIO CORTÁZAR"` choca con Rayuela de Julio Cortázar.)

- [ ] **I7** **3 pts** · Ids que no se reutilizan

Si se crean los libros 1, 2 y 3, se borra el 3 y se crea uno nuevo, el nuevo tiene **id 4**, no 3.

- [ ] **I8** **10 pts** · Persistencia en archivo (obligatoria)

  Los libros se guardan en `datos/libros.json`, dentro de la carpeta del proyecto (junto a `main.py`, **sin importar desde dónde se ejecute**: usá `pathlib`; creá la carpeta `datos` si no existe). Se guarda **después de cada cambio** (crear, reemplazar, actualizar, borrar) y se carga al iniciar. El archivo es una **lista JSON** de objetos con `id`, `titulo`, `autor`, `anio` y `disponible`.
  (3) El archivo se crea con el contenido correcto tras crear libros, y un libro borrado desaparece de él. **(3)** Tras reiniciar la API, los libros siguen y se pueden pedir con `GET /libros` y encontrar con la búsqueda. **(2)** Tras reiniciar, el siguiente id es el mayor guardado + 1. **(2)** Si el archivo no existe, está vacío o está dañado, la API arranca vacía **sin romperse**.

- [ ] **I9** **5 pts** · Búsqueda por texto

  `GET /libros/buscar?texto=…` devuelve **200** con la lista de libros cuyo `titulo` **o** `autor` **contiene** el texto, sin distinguir mayúsculas y minúsculas. **(2)** Encuentra coincidencias en el título. **(2)** En el autor. **(1)** Lista vacía si no hay coincidencias; **422** si falta `texto`. Funciona sobre los datos guardados, también después de reiniciar. *Pista: declarala antes que `/libros/{id}`.*

6 · Condiciones de entrega **10 puntos**

También se comprueban automáticamente, antes que todo lo demás.

- [ ] **E1 (3)** El repositorio se puede **clonar** (es público), `main.py` está en la raíz, se importa y tiene `app`.
- [ ] **E2 (2)** `requirements.txt` existe e incluye `fastapi`.
- [ ] **E3 (2)** `README.md` existe, explica cómo instalar (incluye `pip install`) y cómo correr (incluye `fastapi dev` o `uvicorn`), y tiene al menos 5 líneas con contenido.
- [ ] **E4 (1)** `.gitignore` existe e incluye `.venv` (o `venv`) y `__pycache__`.
- [ ] **E5 (1)** El repositorio **no tiene versionadas** `.venv`, `venv` ni `__pycache__`.
- [ ] **E6 (1)** Al menos **5 commits**, cada uno con un mensaje de 10 caracteres o más (Clase 16).

Si el repositorio no se puede clonar o `main.py` no se puede importar, las tareas fáciles e intermedias valen 0 puntos. El corrector no puede probar lo que no arranca. Probá `python -c "import main"` en la terminal antes de entregar: no tiene que imprimir nada ni quedarse colgado.

7 · Tareas OPCIONALES **hasta +15 puntos**

Desafíos para ir más allá. Suman puntos extra, pero **la nota final no supera el 100%**. Hacelas después de las fáciles e intermedias.

- [ ] **01** **6 pts** · Prestar y devolver

  `POST /libros/{id}/prestar` pone `disponible` en `false` y devuelve **200** con el libro; **409** si ya estaba prestado; **404** si no existe. `POST /libros/{id}/devolver` hace lo inverso: pone `true`; **409** si ya estaba disponible; **404** si no existe. **El cambio también se guarda en el archivo.**

- [ ] **02** **3 pts** · Estadísticas

  `GET /libros/estadisticas` devuelve **exactamente** `{"total": N, "disponibles": N, "prestados": N}` con enteros. *Misma pista: antes que `/libros/{id}`.*

- [ ] **03 (2)** Usa `logging` (al menos 3 llamadas a `logger`) y **no usa `print`**.
- [ ] **04 (2)** Pasa `ruff check` con la configuración por defecto, sin errores.
- [ ] **05 (2)** Todos los endpoints tienen docstring o `summary`.

8 · Cómo se calcula la nota

SECCIÓN	MÁXIMO
Fáciles (F1–F6)	**40**
Intermedias (I1–I9)	**50**
Entrega (E1–E6)	**10**
Opcionales (O1–O5)	**+15**
Nota sobre 100	`mín(100, suma)`
Escala 1 a 10	`nota ÷ 10`

PARA APROBAR

Completar todas las tareas fáciles (F1 a F6).

CRITERIO DE APROBACIÓN · COMPLETA EL PROFE

Nota mínima: _____ / 100 · _____

COMPONENTE MANUAL (OPCIONAL) · COMPLETA EL PROFE

Ej.: breve pregunta oral sobre el código entregado.

_____

Ejemplo: 40 (fáciles) + 36 (intermedias) + 10 (entrega) + 6 (opcionales) = 92 → **9,2**. Otro: 40 + 50 + 10 + 12 = 112 → se corta en 100 → **10**.

9 · Cómo probar tu API antes de entregar

Un guion en `/docs`, en orden. Es casi lo mismo que va a hacer el corrector. Reiniciá el servidor antes de empezar para arrancar con la lista vacía.

1. `GET /` → 200 con el mensaje exacto.

2. `GET /libros` → 200 y `[]`.

3. `POST` dos libros → 201, ids 1 y 2; el primero sin `disponible` → sale `true`.

4. `GET /libros` → los dos, ordenados por id.

5. `GET /libros/1` → 200 el correcto. `/libros/999` → 404. `/libros/abc` → 422.

6. `POST` con titulo `"ab"`, con anio 999, con anio `"abc"` → 422 cada uno. Repetí el libro 1 → 409.

7. `PUT /libros/1` con todo nuevo → 200, mismo id. `PATCH /libros/2` con `{"disponible": false}` → 200, solo eso cambió. `PATCH` con `{}` → 200 igual.

8. `?autor=…`, `?disponible=false`, `?desde=1&limite=1`, y `/libros/buscar?texto=ray` → lo esperado.

9. `DELETE /libros/2` → 204 sin cuerpo. `GET /libros/2` → 404. `DELETE /libros/2` otra vez → 404.

10. `POST` uno nuevo → **id 3**, no 2. **Los ids no se reutilizan.**

```
# y en la terminal, con el venv activo, desde la carpeta del proyecto:
$ python -c "import main"   # no imprime nada, no se cuelga, termina solo
$ ruff check .              # All checks passed! (para 04)
```

GUION DE PERSISTENCIA · I8

P1. Creá 2 libros. Abrí `datos/libros.json`: tienen que estar los dos.

P2. Frená el servidor con **Ctrl+C**. Volvé a levantarlo. `GET /libros` → siguen los 2. `/libros/buscar` los encuentra.

P3. `POST` uno nuevo → id **3** (mayor guardado + 1). Borrá el 1: desaparece del archivo.

P4. Frená. Borrá `datos/libros.json`. Levantá → `[]`. Repetí con el archivo **vacío** y con una **coma de más** adentro: arranca vacía, sin romperse.

También podés probar con un **script de `requests`** (Clase 17), que es exactamente lo que hace el corrector. Revisá que no quede ningún `print` ni `input()`, y que la **URL del repositorio abre en incógnito**.

10 · Plan sugerido · del lunes 9 al lunes 16

Una semana, con **otras clases en el medio**. Por eso conviene avanzar un poco cada día y no dejarlo para el fin de semana. **Commit y push después de cada tarea terminada** (así también cumplís E6).

LUN 9
Ritual: repo en GitHub, venv, .gitignore. Modelo `Libro`. **F1, F2, F3.**

MAR 10
F4, F5, F6. Guion de pruebas de las fáciles completo.

MIÉ 11
I1 a I5: validaciones, PUT, filtros, paginación, PATCH.

JUE 12
I6, I7 y la **persistencia I8**. Guion de persistencia.

VIE 13
Búsqueda I9. README, requirements. Guion completo de pruebas.

SÁB 14 · DOM 15
Opcionales (si lo anterior está completo). Revisión final con el checklist de la sección 12.

LUN 16
Último commit y push. Probar la URL en incógnito. **Completar el formulario antes de las 19:30 hs.**

> **Una hora por día** rinde más que siete el domingo. Si te trabás más de 30 minutos en lo mismo, pedí ayuda con el código y el error completo.

11 · Errores comunes y cómo evitarlos

EL ERROR	QUÉ PASA EN LA CORRECCIÓN	CÓMO EVITARLO
`año` con ñ, `título` con tilde	El corrector busca `anio` y no lo encuentra: falla toda prueba que lo use.	Copiá los nombres de la tabla de la sección 3.
Ids que se reutilizan al borrar (`len()+1`)	Falla I7 y puede romper F6.	Un contador que solo sube (Clase 22).
Rutas fijas después de `/libros/{id}`	`/libros/buscar` da 422: "buscar" no es un entero.	Declará `/buscar` y `/estadisticas` **antes**.
DELETE que devuelve un mensaje	Falla F6: se espera 204 **sin cuerpo**.	`status_code=204` y sin `return`.
PATCH que pisa campos con `None`	Falla I5.	`model_dump(exclude_unset=True)`.
Filtros o búsqueda que distinguen mayúsculas	Falla I3, I6 e I9.	`.lower()` de los dos lados.

EL ERROR	QUÉ PASA EN LA CORRECCIÓN	CÓMO EVITARLO
Ruta del archivo relativa a “desde dónde se ejecuta”	El corrector ejecuta desde otra carpeta: no encuentra el archivo, o lo crea en otro lado. Falla I8.	`Path(__file__).parent / "datos" / "libros.json"` (Clase 15).
Guardar solo al apagar el servidor	El corrector mira el archivo tras cada pedido: falla I8.	Llamá a guardar() al final de POST, PUT, PATCH, DELETE.
No actualizar el contador de ids al cargar	Tras reiniciar, el nuevo id repite uno existente. Falla I8.	Al cargar: contador = mayor id guardado + 1 (o 1 si está vacío).
Archivo vacío o dañado que rompe la carga	La API no arranca: **0 puntos**.	`try/except` alrededor de `json.load` → lista vacía.
Subir `venv/` a GitHub	Falla E4/E5. Repo gigante.	`.gitignore` antes del primer commit.
`print` o `input()` olvidado	`input()` cuelga la importación: **0 puntos**. `print` pierde O3.	Buscá “print(” e “input(” en todo el proyecto.
`main.py` que levanta el servidor solo	La importación nunca termina: **0 puntos**.	Nada de `uvicorn.run` suelto. Se levanta desde afuera.
Todos los commits juntos al final	Falla E6 (5 commits). Y perdés el historial.	Commit después de cada tarea terminada.
Repositorio privado	No se puede clonar: **0 puntos**.	Settings → General → Visibility: Public. Probá en incógnito.
URL del perfil en el formulario	El corrector no encuentra el repo.	La URL termina en el **nombre del repo**, no en tu usuario.

12 · Checklist final antes de entregar

- [ ] Las 6 tareas fáciles pasan el guion de la sección 9.
- [ ] **Probé la persistencia**: Ctrl+C, levantar de nuevo, los libros siguen.
- [ ] `python -c "import main"` termina sin imprimir nada.
- [ ] `main.py` en la raíz · `requirements.txt` · `README.md` (instalar y correr) · `.gitignore`.
- [ ] Hice al menos **5 commits** con mensajes claros.
- [ ] Completé el formulario con DNI, correo, nombre completo y **la URL del repo**.
- [ ] Nombres de rutas y campos **idénticos** al contrato.
- [ ] La API arranca vacía sin archivo, con archivo vacío y con archivo dañado.
- [ ] Sin `print`, sin `input()`, sin `uvicorn.run`.
- [ ] El repositorio es **público** y no tiene venv ni __pycache__.
- [ ] La URL del repositorio **abre en una ventana de incógnito**.
- [ ] Puedo explicar **cada línea** de lo que entrego.

SOBRE EL USO DE IA

Podés usar un asistente de IA como lo practicamos en la **Clase 13**: para entender un error, pedir una explicación o que te revise una función. Lo que entregás tiene que ser **entendido y explicado por vos**: podrías tener que responder preguntas sobre el código. Si la IA te sugiere algo, antes de usarlo preguntale por qué, y comprobalo en /docs. La IA es tu copiloto. El piloto sos vos.

Es tu primera API que va a ser probada por un programa, y tu primer proyecto en GitHub. Que pase todas. 🚀